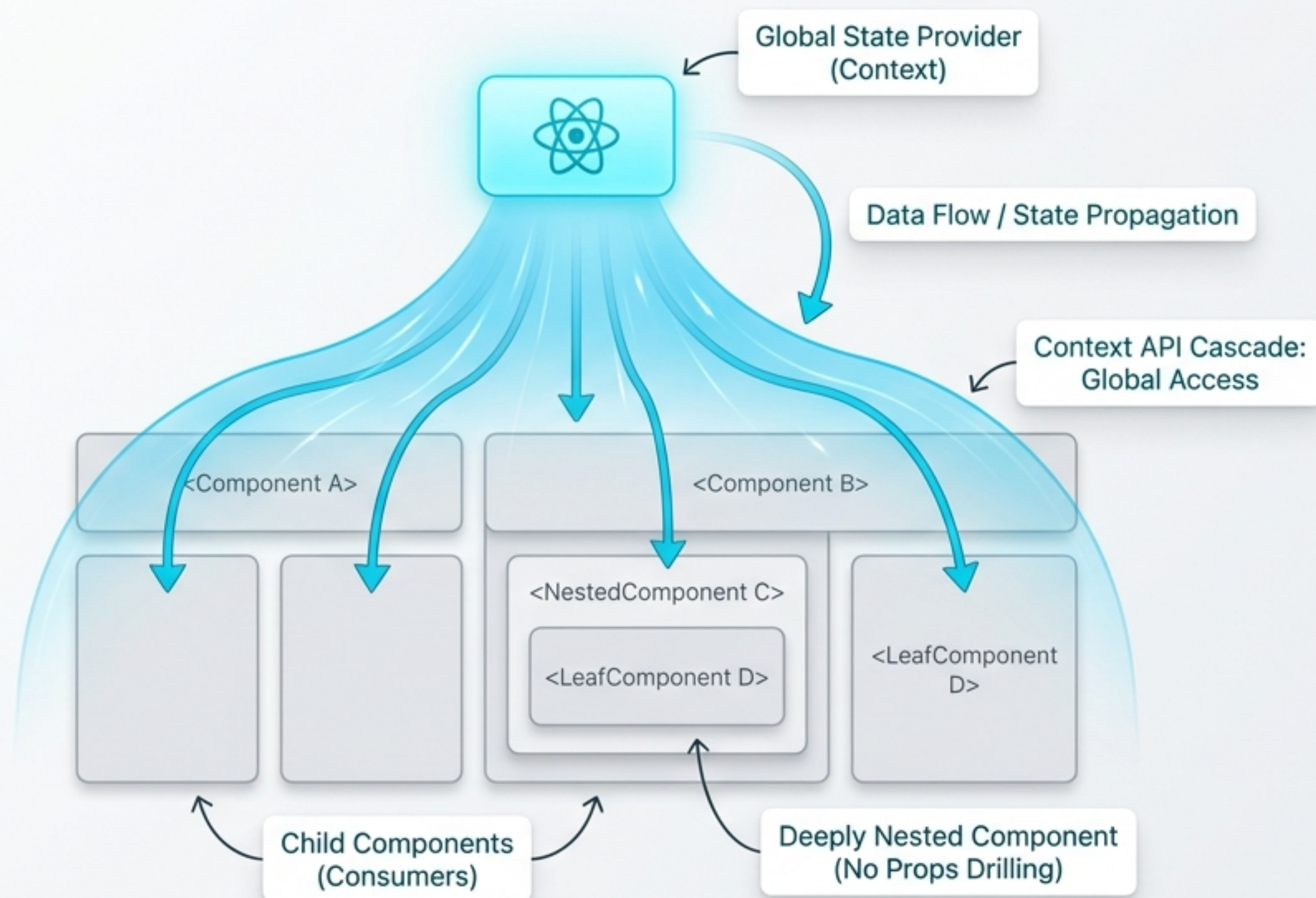


React Native Context API 深度解析

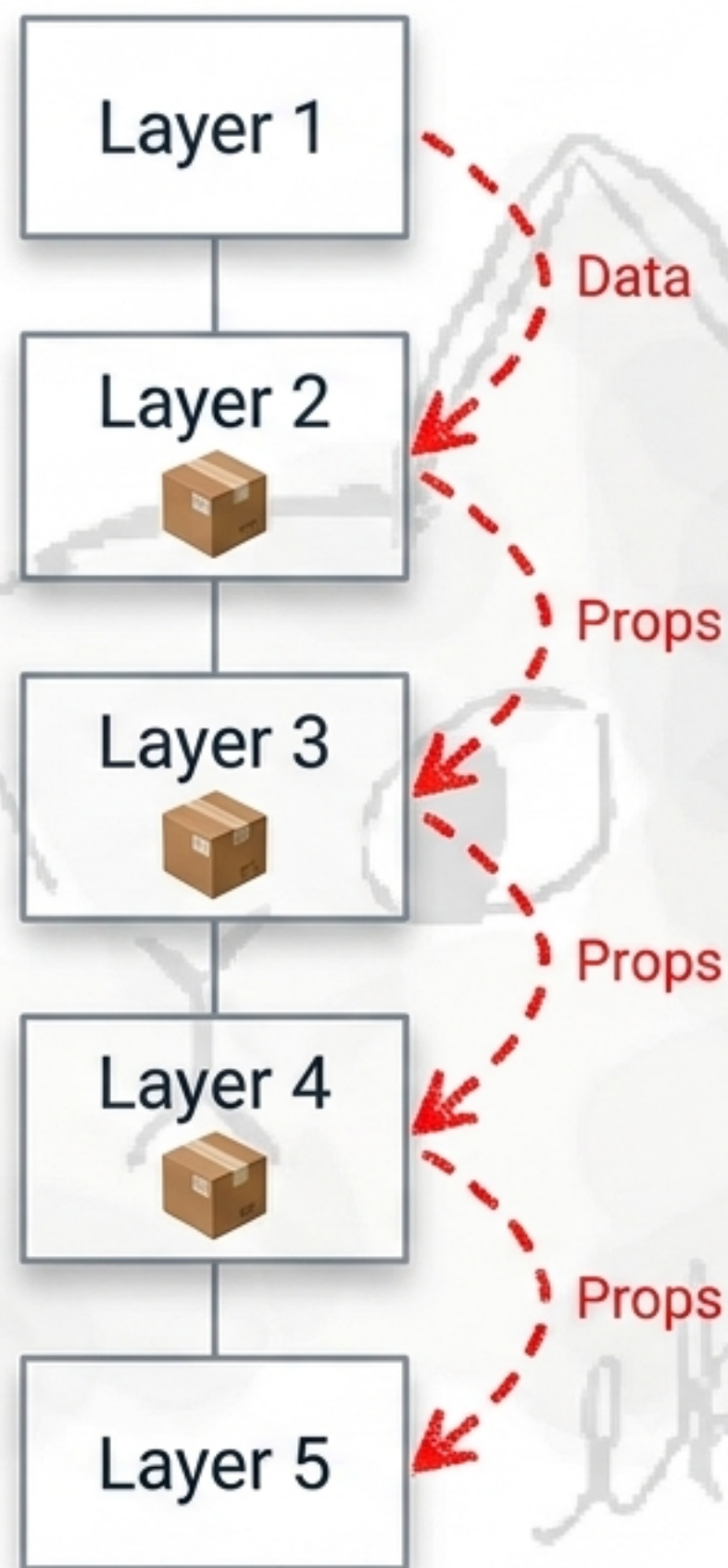
告別 Props Drilling：從核心程式碼掌握全域狀態管理 (Global State)

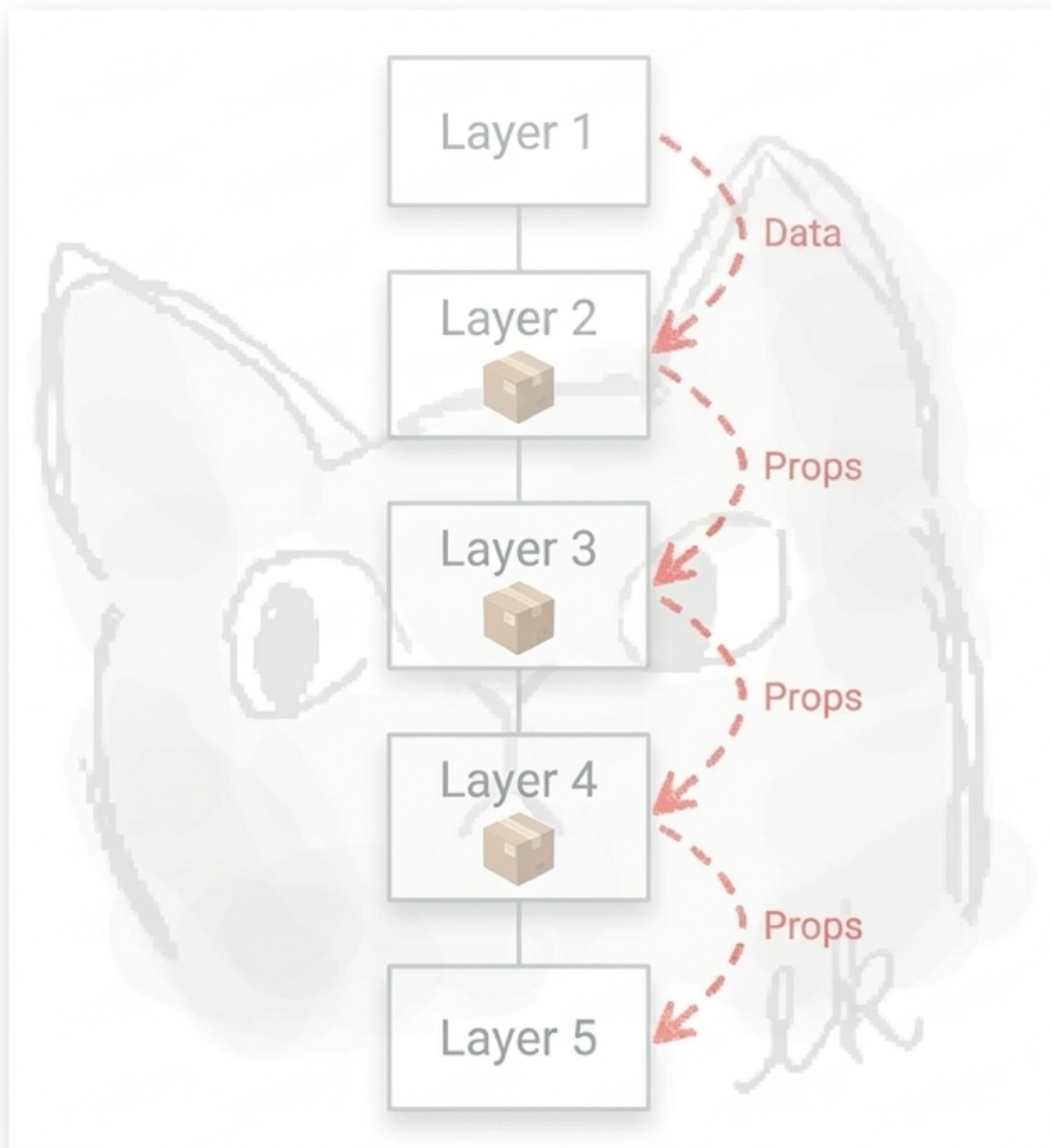


為什麼我們需要 Context ？

痛點：Props Drilling (屬性鑽取)

- 當需要將「登入狀態」或「UI 主題」傳遞到深層元件時。
- 中間層的元件被迫成為「資料快遞員」，即使它們根本不需要這些資訊。
- 結果：程式碼冗長、耦合度過高、難以維護。





Context API：全域狀態的「專屬直達電梯」

Layer 1
Provider

Layer 2

Layer 3

Layer 4

Layer 5
Consumer

在頂層建立一個供應中心 (Provider)。

任何深度的子元件，都能直接「訂閱」並取得資料。

不再需要逐層傳遞 Props！

構成 Context 的三個核心 API



createContext (藍圖)

宣告全新的 Context 物件，定義資料結構與預設值。



Provider (廣播站)

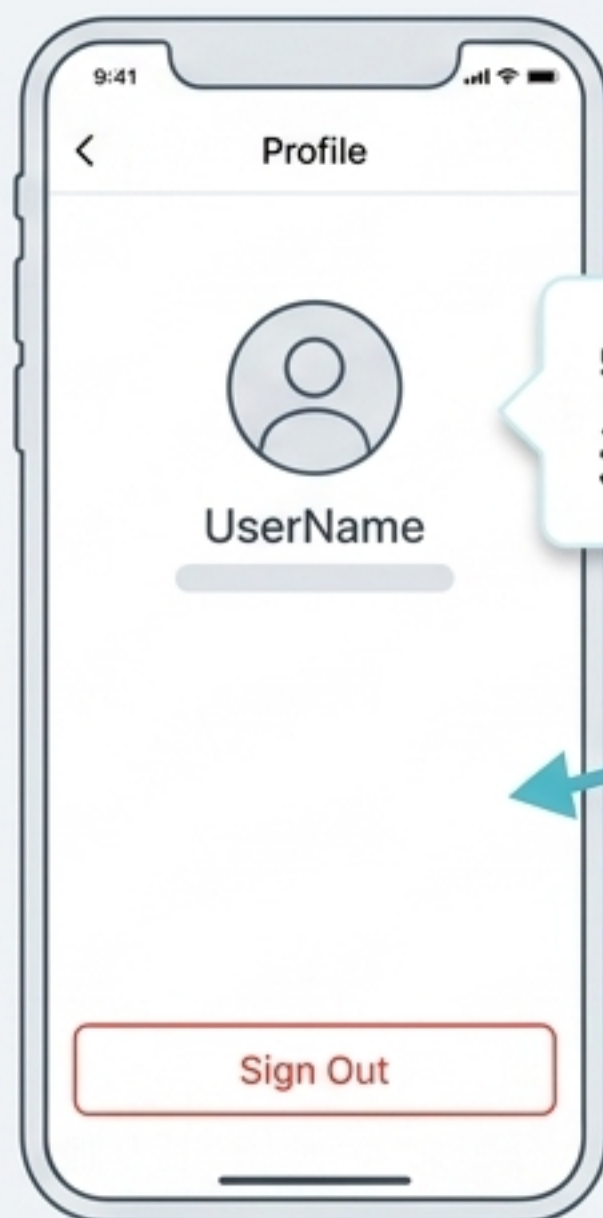
包裹外層元件，負責「提供」真實的全域資料 (value) 給所有子層。



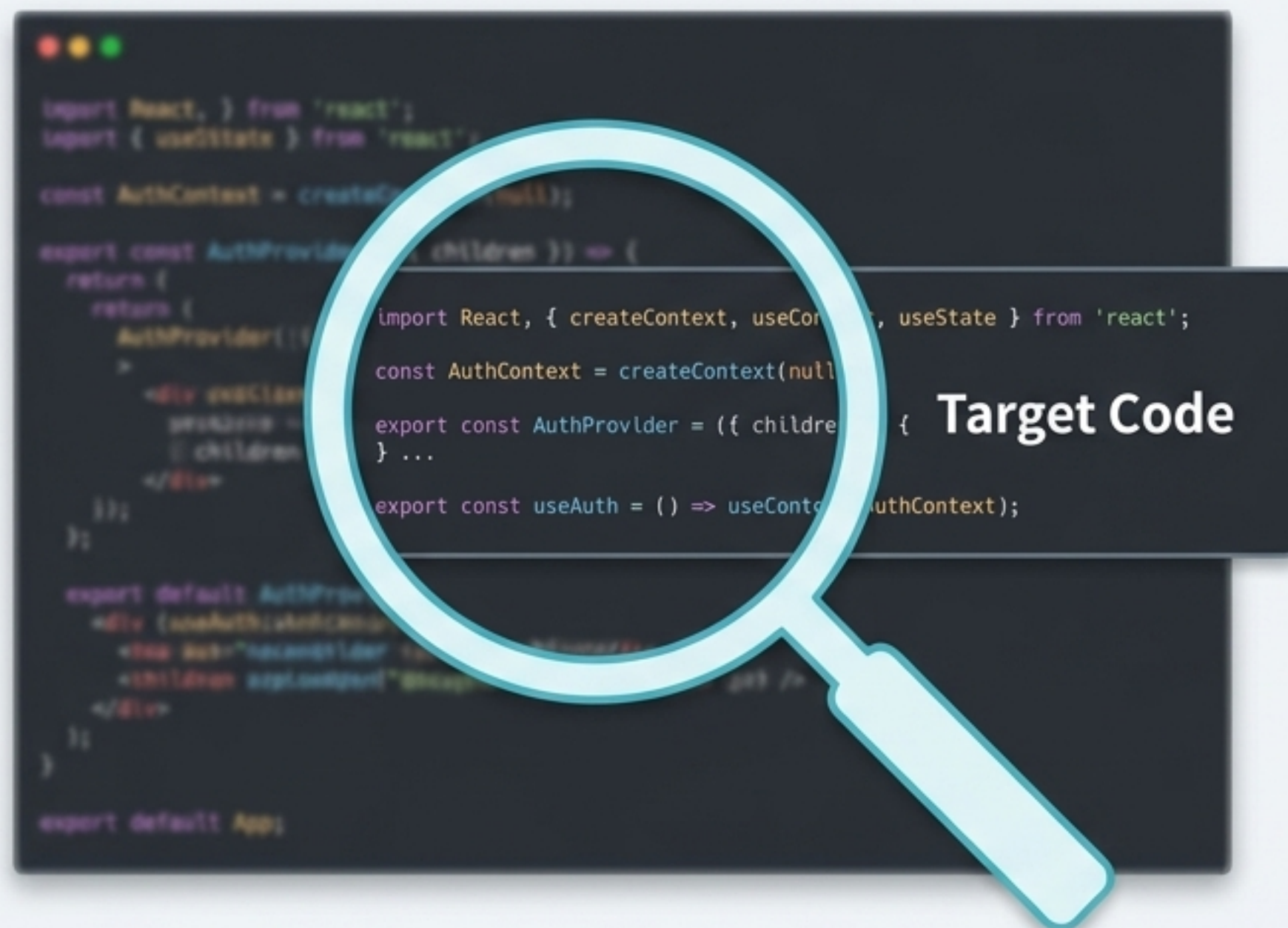
useContext (接收器)

放置於需要資料的子元件內部，負責「讀取」廣播出來的資料。

核心解析：實作「登入帳號」全域狀態



需求：全 App 都需要知道「目前是誰登入」



接下來，我們將這份實戰級的身份驗證 (Auth) 程式碼，拆解為 4 個關鍵步驟進行白板透視。

Step 1: 規範資料結構與建立藍圖

嚴格定義 Context 內部會提供哪些狀態 (user) 與操作方法 (signIn, signOut)。

```
type AuthContextValue = {  
  user: User | null;  
  signIn: (user: User) => void;  
  signOut: () => void;  
  setUser: (user: User) => void;  
};  
  
const AuthContext = createContext(undefined);
```

建立 Context 物件。初始值設為 undefined 是進階防呆機制，若後續在 Provider 外呼叫，可精準觸發報錯。

Step 2: 建立全域狀態管理中心

```
export function AuthProvider({ children }: PropsWithChildren) {  
  const [user, setUser] = useState(null);  
  // ...  
}
```

建立一個包裹元件，接收 **children** (內層的所有畫面與元件)。

核心觀念：所謂的「全域資料」，其實就是定義在頂層 Provider 內的 **useState**！當 **user** 狀態改變時，所有訂閱此 Context 的元件都會同步更新。

Step 3: 使用 useMemo 進行效能優化

```
const value = useMemo(  
  () => ({  
    user,  
    signIn: setUser,  
    signOut: () => setUser(null),  
    setUser,  
  }),  
  [user],  
);
```

依賴陣列：只有當 user 真正改變時，才重新計算並提供新的 value。



為什麼這裡必須用 **useMemo**？
防止不必要的重新渲染（**Re-renders**）！若沒有 **useMemo**，每次 **AuthProvider** 渲染時都會產生全新的物件參考，迫使底層所有子元件無意義地重新渲染。

Step 4: 封裝為自訂 Hook (安全與便利)

```
export function useAuth() {  
  const value = useContext(AuthContext);  
  if (!value) {  
    throw new Error("useAuth must be used inside AuthProvider");  
  }  
  return value;  
}
```

語法糖：子元件只需呼叫 `useAuth()`，不用再每次重複匯入 `useContext` 與 `AuthContext`。



防呆機制 (Fail-fast)：如果開發者忘記在最外層包上 `<AuthProvider>`，因為 Step 1 設為 `undefined`，程式會立刻拋出明確錯誤，大幅降低除錯時間。

將所有零件組合起來 (App 宏觀結構)



實作細節提醒

原始碼最後回傳 `return {children};` 為概念簡寫，實際在 React 運作中，Provider 必須這樣回傳以包裹子元件：
`return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;`

教學現場：如何在畫面中使用？

```
function ProfileScreen() {  
  // 一行程式碼，取得全域狀態與登出函式！  
  const { user, signOut } = useAuth();  
  
  return (  
    <View>  
      <Text>歡迎，{user.name}</Text>  
      <Button title="登出" onPress={signOut} />  
    </View>  
  );  
}
```

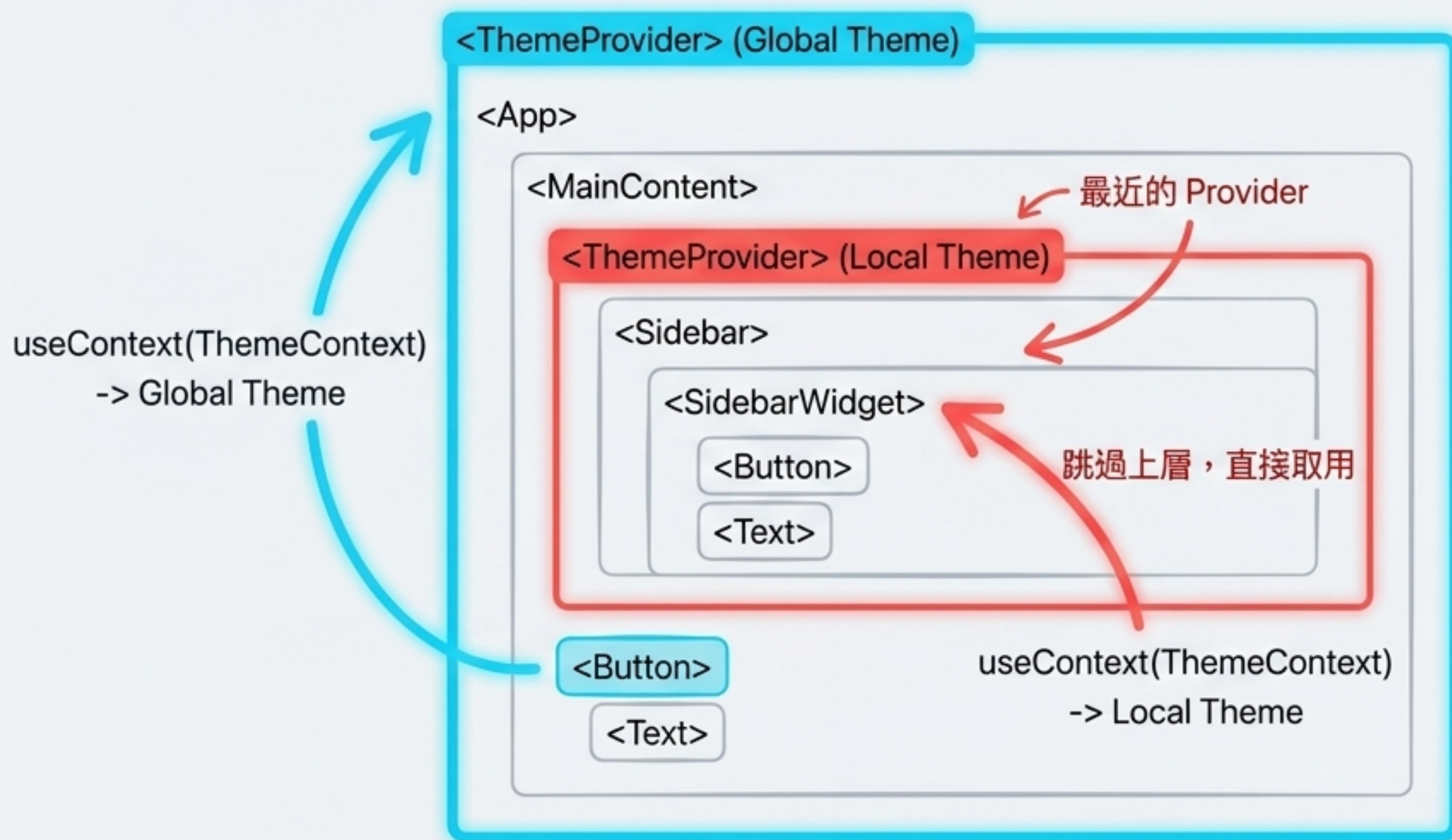
歡迎，Evan

登出

只要在元件的任何地方呼叫自訂 Hook，
就能瞬間「取得」或「改變」全域狀態，
完全不用理會 Props！

Context 運作機制：永遠找「最近的」

如果元件樹中有多個相同的 Provider 怎麼辦？

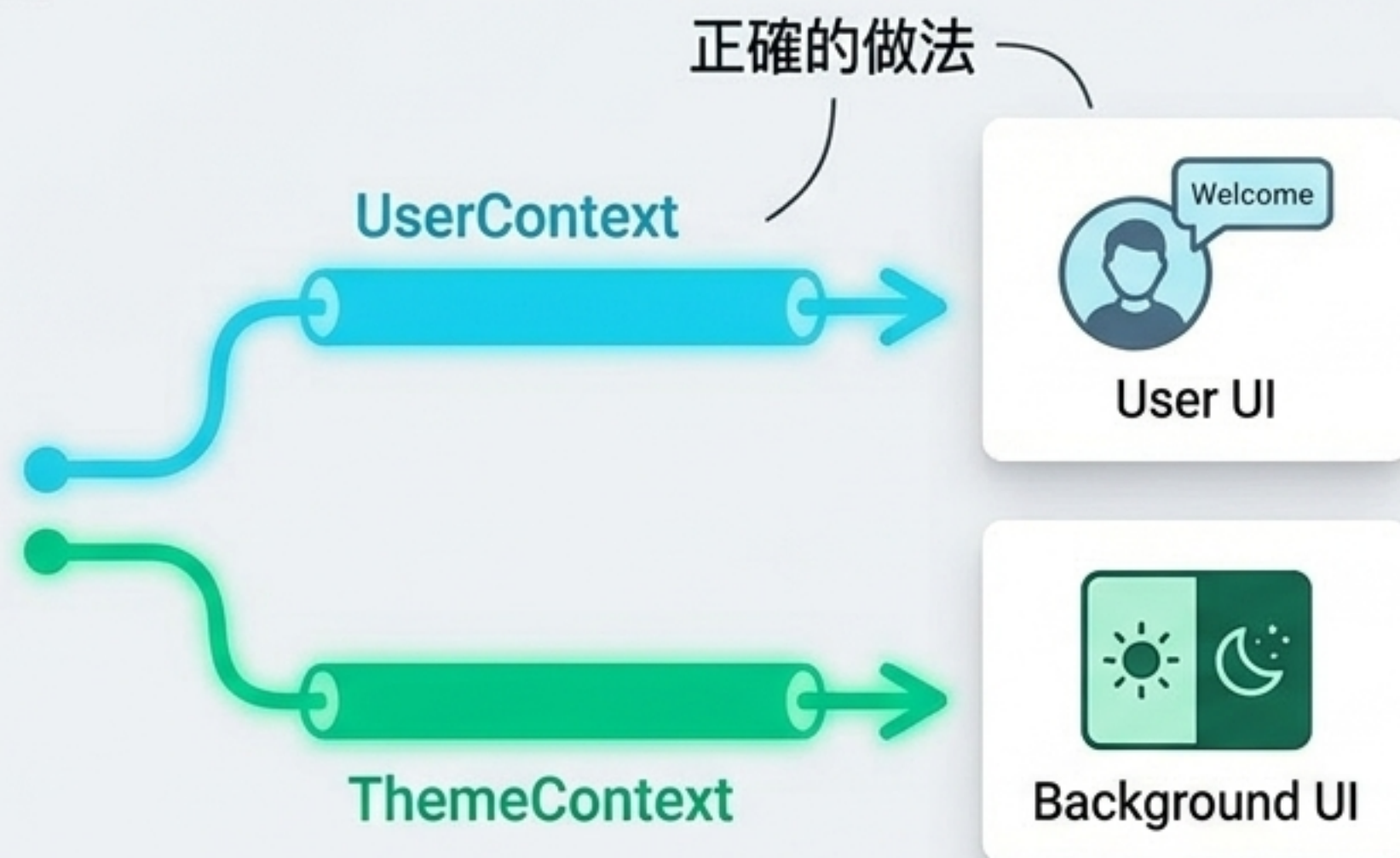


- `useContext` 永遠會往上層尋找距離自己最近的 Provider 取值。
- 應用場景：大部分畫面使用全域設定，但特定區塊（如某個特定面板）可以被第二個 Provider 包裹，套用局部的獨立設定，而不會影響整體。

最佳實務：分離不同的 Context



避免巨無霸 Context：當 Context 中的任何值發生變化時，所有消費者都會重新渲染。



正確架構（關注點分離）：

- **UserContext**：只負責管理登入狀態。
 - **ThemeContext**：只負責管理淺色/深色模式。
- 優勢：依據業務邏輯拆分，提升程式碼可讀性，並完美避免無關的渲染效能浪費。

架構決策：什麼時候「不」該用 Context ？

✓ 適合 Context

- 低頻率更新的全域狀態（如：登入身分 Auth、語系設定 Language、UI 主題 Theme）。
- 需要深入跨越多層元件樹傳遞的資料。

✗ 不適合 Context

- 高頻率更新狀態：如動畫數值、文字輸入框的即時同步（會造成嚴重卡頓）。
- 複雜狀態流動：若狀態邏輯極度複雜，建議改用 Redux 或 Zustand。
- 只是為了解決少數幾層的 Props：層級不深時，應優先考慮「元件組合 (Component Composition)」，將子元件直接當作 Props 傳遞。

知識閉環：Context 資料流總結

